

好东西都是总结出来的，创造 Skill 的 Skill，元 Skill 方法论，全文2w字

Original 西瓜 01fish 2026年3月20日 16:00 上海

道可道，非常道

定位：所有元技能的元技能，贯穿冷启动、迭代、反思、质量控制、柳叶刀方法等所有工序的核心哲学。

附：本篇实践可参考案例为 一个开源的AI《史记》知识库与26个可复用构造知识库Skill，57万字史记，10万个实体、3185个事件、7652条关系全部结构化

全文可视化概览（一页纸）

核心公式

好东西 = 总结出来 ≠ 设计出来

OTF（边做边总结）+ JIT（小步快跑）+ Bootstrap（自举）

知识自动增长，人工投入递减

演进规律：从执行到自治

冷启动：人工执行，0% 自动，产出数据。

模式涌现：边做边总结，20% 自动，产出模式。

知识固化：Agent 学习，60% 自动，产出 Skill。

系统自治：自主运行，90%+ 自动，产出 META。

知识资产：四层复用金字塔

脚本工具：Lint / Extract / Render / Transform，90%+ 主要是微调参数。

项目 Skill：实体标注 / 事件识别 / 消歧，60%-80% 需要适配。

META 技能：反思 / 迭代 / 冷启动，100% 跨项目通用。

元方法：演进式方法论，100% 跨领域通用。

五层抽象：知识压缩过程

层1 案例：具体错误、修正记录。

层2 模式：错误模式、工具模式。

层3 原则：OTF + JIT + Bootstrap。

层4 洞察：总结是唯一的知识来源。

层5 哲学：演进 > 设计。

三个反常识

不是先设计完美方案再执行，而是先执行最小版本，边做边总结优化。

不是等项目结束再总结，而是每轮迭代都总结，知识实时沉淀。

不是追求一次性完美，而是快速交付 60% 质量，再迭代收敛到 95%。

阅读导航

零、核心命题：3 分钟快速理解。

一、通用方法论框架：OTF + JIT + Bootstrap 详解。

二、五层抽象金字塔：如何提炼知识。

三、为什么总结：认知科学与信息论基础。

四、实战案例：史记项目全流程复盘。

五、Agent 学习路径：如何让 AI 自主总结。

零、核心命题

一句话

好东西都是总结出来的，不是发明出来的。

核心方法论：OTF + JIT + Bootstrap

OTF = On-The-Fly，即时总结

不是做完再总结，而是边做边总结。

发现模式的那一刻，就立即记录、归纳、应用。

总结不是项目结束时的附加动作，而是执行过程的一部分。

JIT = Just-In-Time，即时交付

不是一次性做完美，而是及时交付最小可用版本。

先跑起来，再拿反馈，再改进，再收敛。

重点不是“第一次就对”，而是“每一轮都更对”。

Bootstrap = 知识自举

每一步迭代，都会产生下一步可复用的元知识。

知识会在迭代中自我增殖，自动化程度递增，人工干预递减。

一开始是人带系统跑，后面是系统反过来加速人。

OTF + JIT + Bootstrap = 自举式知识增殖系统

维度	传统方式	OTF + JIT
总结时机	项目结束后（Post-Mortem）	实时/每轮（On-The-Fly）
交付节奏	大批量一次性（Big Bang）	小批量高频次（Just-In-Time）
质量策略	追求首次完美	快速迭代逼近完美
知识积累	项目结束才总结	每轮迭代都总结
风险控制	后期集中爆发	早期快速暴露

三个推论

- 1. **知识是压缩**
— 从10,000个案例→100条模式→10条原则→1个洞察
- 2. **理解是归纳**
— 看到规律才算真正理解，没有规律就没有知识
- 3. **方法是沉淀**
— 做一次是经验，做十次总结出来才是方法论

演进式方法论总纲

一、基本原则

演进 > 设计：多总结，少设计。

聚焦 > 全面：一次只做一件事。

实践 > 理论：好东西都是总结出来的。

二、操作层次

- 观察现象：先记录案例。
- 识别关键：判断什么最影响结果。
- 提取模式：从重复现象里发现规律。
- 固化方法：把规律写成文档、脚本、Skill。

三、迭代策略

- 小粒度：从小问题开始。
- 小工具：优先解决最痛的重复劳动。
- 小周期：快速反馈，快速迭代。

四、价值导向

- 防止重复犯错：把隐性知识显性化。
- 降低认知负荷：把方法标准化。
- 提高复用效率：让模式可以迁移。

所有元技能清单

本项目最终形成**14个元技能**（13个方法论+1个元元技能）。

核心元技能（本文涉及）：

编号	元技能	核心理念	与本文档关系
00-META-00	好东西都是总结出来的	OTF+JIT+Bootstrap 自举增殖	本文档 (元元技能)
00-META-01	反思	Agent自主发现错误模式	反思 = 自动化的OTF总结
00-META-02	迭代 workflow	广度优先、模式驱动、指数衰减	每轮总结错误模式表
00-META-03	冷启动	5个样本建立直觉	快速总结初步模式
00-META-04	柳叶刀方法	精细分解、混合驱动、成本意识	混合方案矩阵是总结出来的
00-META-07	可读性	中间产物人类可读、Git友好	总结的前提是数据可读
00-META-08	标注体系设计	本体从数据中"长"出来	本体演化是总结的过程

编号	元技能	核心理念	与本文档关系
00-META-09	Agent提示词工程	SKILL作为Agent的"DNA"	所有 SKILL 都是总结出来的
00-META-10	质量控制	Lint规则来自错误案例	错误案例→总结→规则
00-META-11	数据体感培养	多维观察数据异常	观察→发现→总结

注：另有00-META-05（知识作为上下文压缩）、00-META-06（SKILL优化与演化）、00-META-12（数据融合）、00-META-13（技能迁移）共14个元技能。

层级关系：

元元技能

本文档讨论的是“如何总结”本身，也就是关于总结的总结。

往下一层，是方法论元技能

迭代 workflow、冷启动、柳叶刀方法、标注体系设计、可读性、质量控制。
它们回答的是：复杂知识工程任务，应该怎么做。

再往下一层，是执行层元技能

实体标注、事件识别、消歧、渲染、发布、数据融合、技能迁移。
它们回答的是：具体任务，实际怎么落地。

核心关系

所有元技能都不是预先设计出来的，而是从大量任务里总结出来的。
每个元技能都遵循同一套抽象路径：案例 → 模式 → 原则 → 洞察 → 哲学。
本文档不是替代其他元技能，而是解释这些元技能为什么会长成现在这样。

核心关系： - 所有8个元技能都是通过**总结**提炼出来的（而非预先设计） - 每个元技能的形成都遵循**五层抽象金字塔**（案例→模式→原则→洞察→哲学） - 本文档（元元技能）是对“如何总结”的递归总结

一、通用方法论框架

本章导航：本章分为两部分 1. **通用原则**（1.1-1.6节）：从多年实践中抽象出的知识处理方法
论 2. **实践机制**（1.7-1.8节）：OTF + JIT 作为方法论的具体落地机制

1.1 总结的双重目标

目标1：决策优化 — 在实践中检验逻辑，建立优先级的深刻共识

总结有两个目标：一是理解更深，二是方法更稳。

目标1：决策优化

理解深度不是停在 Framework，而是继续追到 Rationale，最后长出 Insight。

Framework 解决“怎么做”，Rationale 解决“为什么这样做”，Insight 解决“下一次还会发生什么”。

目标2：方法沉淀

沉淀路径不是一句空话，而是清晰的五级演化：案例 → 方法 → 流程 → 制度 → 文化。

一次是经验，三次可以总结为方法，十次才能稳定成流程。

实践方法： - 每件事都追问：为什么做？为什么不做其他？ - 记录决策过程，而非只记录决策结果 - 周期性回顾：这个决策的rationale现在还成立吗？

目标2：方法沉淀 — 将有效方法固化为可复用模式

实践方法： - 一次是经验，三次总结为方法，十次固化为流程 - 方法文档化：写成SKILL文档、检查清单、自动化脚本 - 定期审查：哪些方法已失效？哪些需要更新？

1.2 知识提炼的四个层次

基于“复盘四步法”抽象出的通用框架：

1. 观察层：记录现象

- 做了什么？结果如何？
- 数据是什么？异常在哪里？

2. 分析层：识别关键

- 哪一步是关键步骤？
- 哪个因素影响最大？

3. 归纳层：提取模式

- 有哪些重复规律？
- 其他案例是否支持这个规律？

4. 演绎层：形成方法

- 如何应用到新场景？
- 边界条件是什么？

核心价值： - 防止Agent在同一个地方犯错 - 将隐性知识显性化（人类执行经验 → Agent可读的SKILL） - 从单次执行提升为可复用的Agent知识资产

实践案例（抽象版本）：

观察：处理了130章数据，发现13,000个错误

↓

分析：60%是消歧问题，这是关键瓶颈

↓

归纳：消歧错误遵循25个模式（如"同名不同人"）

↓

演绎：开发自动检查工具，规则覆盖80%的情况

1.3 创新的边界与节奏

聚焦原则：一次只解决一个核心问题

失败模式："换四个引擎"

同时改变数据模型 + 处理工具 + 工作流程 + 评价标准

- 无法定位问题根源
- 认知负荷过大
- 反馈周期过长

成功模式："单点突破"

只改变数据模型，工具/流程/标准保持不变

- 快速验证效果

- 明确归因
- 容易回滚

实践建议： - 每个迭代周期只改变一个变量 - 如果必须同时改变多个，至少要能清晰解耦 - 警惕"全面革新"的诱惑

认知负荷管理

降低学习成本的三个策略：

1. 渐进式复杂化
 - 先用简单方案建立直觉
 - 再逐步引入复杂特性
2. 利用现有知识
 - 新工具尽量与旧工具接口兼容
 - 新方法尽量基于现有方法演化
3. 分层隐藏细节
 - Agent初始阶段只需了解最简模型
 - Agent成熟后才需要了解全部细节

短周期迭代

对比：

瀑布模式：规划3个月 → 开发2个月 → 测试1个月 → 发现问题回到起点

迭代模式：规划3天 → 开发2天 → 测试1天 → 调整再来

优势：

- 快速反馈 > 完美规划
- 真实数据 > 理论推导
- 早期暴露问题 > 后期集中爆发

1.4 知识工作的"小"策略

小粒度：从小问题开始，避免过早抽象

错误路径：

Day 1 → 设计通用本体框架

Day 10 → 发现框架不适用于实际数据

Day 20 → 推翻重来

正确路径：

Day 1 → 处理5个具体案例

Day 2 → 归纳初步模式

Day 3 → 处理30个案例验证

Day 5 → 提炼通用框架

原则： - 先有具体的"语义"，再有抽象的"语义网" - 先解决特定问题，再提炼通用方案 - 容忍局部不一致，在演化中逐步收敛

小工具：降低工具使用门槛

工具演进的三个阶段：

1. 人工操作（人类执行，Agent观察学习）
2. 半自动脚本（固化重复步骤，Agent辅助执行）
3. 全自动工具（Agent自主执行，人类审查）

关键：

- 工具演进 = Agent能力演进
- 好工具来自真实执行过程的痛点
- 充分利用现有工具生态（不要重新发明轮子）

小周期：精益迭代，减少浪费

懒处理策略：

- 需要时再处理（不是提前处理所有可能情况）
- 高频问题优先（80%的价值来自20%的问题）
- 自动化ROI分析（工具开发成本 vs 节省时间）

Build-Measure-Learn循环：

1. Build：最小可用版本（1天）
2. Measure：实际使用反馈（1周）

3. Learn: 总结问题和改进点 (1天)

→ 重复

1.5 演进优于设计

核心原则："多总结，少设计"

设计思维：

- 预先规划完美方案
- 追求一次性正确
- 标准化、规范化

演进思维：

- 从现状出发小步改进
- 容忍多样性、试错
- 在竞争中自然选择

为什么演进更有效？

1. 未来需求不可预测

- 设计时无法考虑所有场景
- 演进可以根据实际需求调整

2. 认知能力有限

- 很难一次性设计出完美方案
- 通过迭代逐步逼近最优解

3. 环境持续变化

- 今天的完美方案，明天可能过时
- 演进保持系统的适应性

实践策略：允许"重新发明轮子"

传统观点：

- 避免重复造轮子

- 统一标准、统一工具

演进观点：

- 百花齐放，在竞争中进化
- 不同场景可能需要不同的"轮子"
- 最终收敛到最优方案（自然选择）

渐进式改进

三个原则：

1. 在现有基础上演化
 - 充分利用现有工具/方法
 - 不要从零开始
2. 每个阶段有自身核心任务
 - 不要试图一步到位
 - 阶段性目标要明确
3. 总结优于规划
 - 从实践中提炼模式
 - 多观察、多实验、多总结

1.6 方法文档化的价值

文档化循环

文档不是结项纪念册，而是持续工作的反馈回路。

实践先发生。

在实践中发现重复步骤和异常边界。

发现模式之后，把它写进文档。

文档再反过来指导新实践。

新实践又暴露出新边界，再推动文档更新。

这就是文档化循环：实践 → 发现模式 → 写入文档 → 指导新实践 → 再更新。

所以文档不应该项目结束后一次性写完，而应该随项目一起演进。

文档的两种形式

规范性文档：

- "应该怎么做"
- 示例：标注规范、质量标准

描述性文档：

- "为什么这样做"
- 示例：决策历史、演化过程

两者都重要：

- 规范性：指导执行
- 描述性：传递理解

元认知：容忍多样性

洞察："世界观的attention"

- 同一问题可能需要多个schema
- 不同的人/场景可能有不同的理解
- 不强求统一，在使用时选择/对齐

实践：

- 允许多个标注体系并存
- 在应用层做映射/转换
- 演化中逐步收敛（而非强制统一）

1.7 实践机制：OTF（On-The-Fly）即时总结

定位：以下1.7-1.8节介绍OTF + JIT作为上述通用方法论的具体落地机制

核心理念：不等做完再总结，而是**发现模式的那一刻立即总结**

为什么OTF有效？

认知科学依据： - **记忆衰减** — 做完130章再总结，前面的细节已遗忘 - **模式识别** — 人脑擅长识别模式（看到3次重复就能归纳） - **即时强化** — 总结后立即应用，形成正反馈循环

传统方式（Post-Mortem）的问题：

Post-Mortem 的问题

Day 1-90 一直执行，不总结。

Day 91 才开始回头整理。

结果通常是：细节忘了，总结质量低；错误已经重复了 90 次，返工成本高；总结也很难反哺项目本身，因为项目已经做完。

On-The-Fly 的做法

Day 1 做 5 个样本，就开始总结。

Day 2 把总结出来的规则反过来用于新样本。

Day 3-10 持续边做边改，错误率在执行中就开始下降。

OTF方式 (On-The-Fly)：OTF 的基本工作方式是：执行中发现模式，立即长出规则，再把规则反向喂回执行。

OTF的三个时机

时机1：发现重复（3次规则）

OTF触发器逻辑：- 维护错误模式的重复计数器 - 对每个任务，检测执行结果是否为错误 - 如果是错误，识别其模式并累加计数 - 当某个模式重复 ≥ 3 次时：- 立即总结该模式 - 将总结的规则写入规则库 - 记录OTF总结日志

时机2：发现异常（意外情况）

案例：标注时发现"周公"既是人名又是官职

传统：继续标注，等做完再处理

OTF：

- └─ 立即停下（5分钟）
- └─ 分析：需要新分类"兼职"（如"周公"= 人名+官职）
- └─ 更新标注体系
- └─ 继续标注（后续不再遇到此类困惑）

时机3：批次边界（每N个样本）

每10个样本 → 微总结（5分钟）

- 发现了什么模式？
- 需要调整什么规则？

每50个样本 → 中总结（30分钟）

- 哪些模式高频（写入自动化脚本）
- 哪些模式低频（记录但不自动化）

每130个样本 → 大总结（2小时）

- 形成完整错误模式表
- 提炼方法原则

本项目的OTF实践

实体标注的OTF过程：

Day 1（5章）：

OTF-1: 发现"武王"需消歧 → 规则："谥号必须加邦国"

Day 2（30章）：

OTF-2: 发现"邦国"≠"地名" → 本体演化：4类→6类

Day 5（130章）：

OTF-3: 发现"官职"≠"爵位" → 本体演化：6类→8类

Day 10（第一轮反思）：

OTF-4: 发现系统性消歧遗漏 → 开发自动检查工具

Day 15（第二轮反思）：

OTF-5: 发现格式不一致 → 开发Lint工具

每次OTF的产出： - 规则更新（立即应用于后续样本） - 本体演化（本体是"长"出来的） - 工具开发（自动化重复劳动）

1.8 JIT（Just-In-Time）：及时交付的威力

核心理念：不追求一次完美，而是**及时交付最小可用版本，快速迭代**

为什么JIT有效？

工程经验依据： - **需求不确定** — 前期对"好"的标准认知模糊 - **反馈驱动** — 只有交付了才能获得反馈 - **风险可控** — 小批量迭代比大批量返工成本低

传统方式（Big Bang）的问题：

Week 1-4: 设计完美方案（50属性本体）
Week 5-8: 一次性执行（发现方案不适配）
Week 9: 大规模返工（本体重构，数据重录）

→ 总耗时9周，质量未必高（前期决策错误）

JIT方式（Just-In-Time）：

Week 1:
JIT-v0.1: 最小本体（4类）+ 标注5章
└─ 交付：初版数据（质量60%）
└─ 反馈：需要"邦国"类

Week 2:
JIT-v0.5: 本体演化（4类→8类）+ 标注30章
└─ 交付：扩展数据（质量75%）
└─ 反馈：需要"器物"类

Week 3:
JIT-v1.0: 本体稳定（18类）+ 标注130章
└─ 交付：全量数据（质量85%）
└─ 反馈：系统性错误模式

Week 4:
JIT-v1.5: 第一轮反思修正
└─ 交付：高质量数据（质量92%）
└─ 反馈：收敛

→ 总耗时4周，质量92%（渐进收敛）

JIT的三个层次

层次1：功能JIT（最小可行产品）

不是：设计完整工具链（Lint + Validate + Transform + Export）
而是：

Day 1: 需要格式校验 → 开发 lint_format.py (30分钟)

Day 5: 需要统计分析 → 开发 stats.py (1小时)

Day 10: 需要导出功能 → 开发 export.py (2小时)

→ 按需开发 (JIT)，无冗余功能

层次2：质量JIT（梯度收紧）

不是：所有数据都要求95%准确率

而是：

P0: 60%准确率（2天完成130章）

P1: 80%准确率（1周）

P2: 90%准确率（1周）

P3: 97%准确率（1周）

→ 及时交付可用版本，逐轮提升质量

层次3：知识JIT（按需沉淀）

不是：项目结束后写一份完整文档（50页）

而是：

Week 1: 冷启动文档（3页，够用）

Week 2: 本体设计文档（5页，基于实际演化）

Week 4: 反思方法文档（8页，基于第一轮经验）

Week 8: 完整META技能文档（20页，基于全流程总结）

→ 文档也是JIT，随项目演化

本项目的JIT实践

事件年代标注的JIT过程（真实时间线）：

P0（2小时）：

交付：3,092事件初版年份（准确率60%）

方法：LLM快速标注全部事件

质量目标：能用即可，快速覆盖

P1（半天）：

交付：第一轮反思修正（1010处，准确率80%）

方法：Agent自动反思 + 批量修正

质量目标：系统性错误消除

P2（2小时）：

交付：第二轮反思修正（431处，准确率88%）

方法：基于P1模式库的增量修正

质量目标：模式驱动修正

P3（2小时）：

交付：第三轮反思修正（465处，准确率92%）

方法：跨章一致性检查

质量目标：跨章一致性

P4（1小时）：

交付：第四轮反思修正（167处，准确率95%）

质量目标：接近收敛

P5（1小时）：

交付：第五轮反思修正（46处，准确率97%）

质量目标：收敛完成

总耗时：约1.5天（vs 一次性完美需要1周+）

JIT的收益： - 2小时即可交付可用版本（P0，60%质量） - 避免一次性完美的返工风险 - 每轮都有可交付成果（持续价值） - 总时间反而更短（1.5天 vs 1周+）

1.9 OTF + JIT 的协同效应

单独OTF的问题： - 总结太频繁 → 执行效率低 - 总结质量受限于样本量

单独JIT的问题： - 迭代无方向 → 可能震荡不收敛 - 缺乏方法积累 → 每轮都是新探索

OTF + JIT 协同：

JIT 提供大循环，OTF 提供小循环。

JIT 的节奏是：P0 → P1 → P2 → P3 → 收敛。

OTF 的节奏是：每一轮内部持续发现模式、总结模式、应用模式。

所以真实协同关系是：

P0 阶段先交付 60% 的版本。

在 P0 内部，通过 OTF 长出第一批规则、本体调整 and 工具。

P1 再把 P0 的总结结果吃进去，系统性修正。

后面每一轮，都建立在前一轮沉淀出的模式之上。

JIT 提供框架，OTF 提供燃料。

JIT 负责把版本往前推，OTF 负责把知识往上提。

具体协同：

P0阶段（JIT框架）：

Day 1: 标注5章

└─ OTF-1: 发现消歧模式 → 更新规则

Day 2: 标注30章

└─ OTF-2: 发现分类模式 → 本体演化

Day 3-4: 标注130章

└─ OTF-3: 发现格式模式 → 开发Lint

Day 5: P0交付（60%质量，25条OTF总结的模式）

P1阶段（JIT框架）：

应用P0的25条模式（OTF产出）

└─ 系统性修正1010处

└─ OTF-4: 发现新模式1条

└─ P1交付（80%质量，26条累计模式）

→ JIT提供框架，OTF提供燃料（模式）

复利效应： - OTF总结的模式 → JIT迭代的基础 - JIT交付的版本 → OTF总结的样本 - 两者相互促进，形成正反馈

二、Bootstrap：知识自举的五阶演化

2.1 核心理念：知识增殖与自动化递进

传统观念（错误）：

传统观念

人工劳动 → 完成任务 → 项目结束 → 下个项目从头再来。

自举观念

人工劳动会产生中间知识，比如日志、脚本、检查规则。

中间知识会进一步总结成 Skill。

Skill 会再长出自动化工具。

工具使用过程又会产生更高阶的模式。

最后，系统开始具备自动总结、自动演化的能力。

自举理念（正确）：Bootstrap 不是重复跑流程，而是每一轮都长出更高阶知识。

核心洞察： - 每一步迭代都是**下一步的燃料**（知识增殖） - 自动化程度**指数增长**（从 0%→20%→60%→90%→99%） - 人的角色**从执行者→监督者→设计者**

2.2 第一阶：人工劳动 + 记录（Week 1）

状态： - 自动化率：0% - 人工占比：100% - 中间产物：聊天记录、操作日志、临时笔记

典型场景：

Day 1-2: 手动标注5章

└─ Claude对话：30轮问答，探索标注规范

└─ 产出数据：5章标注文件

└─ 副产品：

- 30轮对话记录（包含决策过程）

- errors.txt（记录了12个犯错案例）

- questions.md（3个未解决的模糊问题）

关键点： - **不要丢弃中间过程**（对话、日志、草稿） - 这些“废料”是下一阶段的“原料” - 人工阶段已经在积累“隐性知识”

2.3 第二阶：脚本化 + 局部自动化（Week 2）

状态： - 自动化率：20% - 人工占比：80% - 中间产物：检查脚本、统计脚本、格式转换脚本

典型场景：

Day 3-4: 标注30章，发现重复劳动

└─ 人工：标注（80%时间）

└─ 发现模式：

- 每章结束都要手动统计实体数 → 写 count_entities.py（10分钟）
- 每次提交前都要检查格式 → 写 lint_format.py（30分钟）
- 每次都要生成markdown索引 → 写 generate_index.py（1小时）

Day 5-7: 使用脚本标注30章

└─ 人工：标注（60%时间，↓20%）

└─ 脚本：自动检查/统计/生成（20%时间，但只需运行命令）

└─ 节省时间：20% → 用于标注更多章节

关键进展： - 从“纯手工”到“脚本辅助” - 脚本是**第一层知识固化**（从隐性→显性） - 但脚本还需人工触发（半自动化）

2.4 第三阶：SKILL 总结 + workflow自动化（Week 3-4）

状态： - 自动化率：60% - 人工占比：40% - 中间产物：SKILL文档、Agent反思workflow

典型场景：

Week 3: 完成130章初版，回顾30个脚本

└─ 发现：脚本之间有共同模式

└─ 总结：写 SKILL_02_实体标注反思.md

└─ 记录：18类实体的识别规则

└─ 记录：12条常见错误模式

└─ 记录：脚本使用的最佳顺序

└─ 自动化升级：

- 写 workflow_entity_reflect.sh（串联所有脚本）
- Agent读取SKILL → 自主执行反思流程
- 人工只需：启动workflow + 审查结果

Week 4: 第一轮反思，Agent自动修正1010处

- └─ 人工：启动Agent + 审查（10%时间）
- └─ Agent：读SKILL → 自动识别问题 → 自动修正（90%时间）
- └─ 新产出：
 - 反思报告（Agent自动生成）
 - 新发现的25条错误模式（Agent总结）

关键进展： - **SKILL = 工作流的"DNA"**（可被Agent读取和执行） - 从"人写脚本"到"Agent读SKILL自动执行" - 人从"执行者"变为"审查者"（人工占比从100%→40%）

2.5 第四阶：自动总结 SKILL（Week 5-8）

状态： - 自动化率：90% - 人工占比：10% - 中间产物：自动生成的SKILL更新、模式库自动扩展

典型场景：

Week 5: 第二轮反思，发现新模式

- └─ Agent执行反思（自动）
- └─ Agent发现新模式（自动）
- └─ ****Agent自动更新SKILL**（新！）**
 - 读取 SKILL_02_实体标注反思.md
 - 检测到新模式P26: "确定性分层不清晰"
 - 自动追加到SKILL文档：


```
...

### 模式P26: 确定性分层不清晰
- 发现于：第二轮反思（2026-03-10）
- 频率：431处
- 修正方法：明确"确定"/"推定"/"估算"三级
...

- 人工只需审查是否合理
```

Week 6-8: 连续三轮反思

- └─ Agent读取不断更新的SKILL
- └─ 每轮自动总结新模式（如果有）
- └─ 自动更新SKILL → 下轮自动应用
- └─ 人工：每轮只需10%时间审查

关键进展： - 总结本身自动化了！ - SKILL从"人写"变为"Agent写+人审" - 知识增殖进入自循环 (SKILL → Agent → 新SKILL → Agent')

2.6 第五阶：元进化（进化的进化）（Week 9+）

状态： - 自动化率：99% - 人工占比：1%（只管理异常情况） - 中间产物：元SKILL（关于如何总结SKILL的SKILL）

典型场景：

Week 9-10: 跨章节事件关系提取

- └─ Agent读取 SKILL_05_事件关系提取.md
- └─ 发现：这个SKILL和 SKILL_02_实体标注 有共同模式
- └─ **Agent自动提炼元SKILL**:
 - 创建 00-META_迭代 workflow.md
 - 提炼通用原则：广度优先、模式驱动、指数衰减...
 - 这些原则可应用于任何反思任务

Week 11+: 新任务（战争事件提取）

- └─ Agent读取 00-META_迭代 workflow.md（元SKILL）
- └─ Agent自动生成任务专用SKILL：SKILL_05e_战争事件识别.md
 - | - 基于元SKILL的模板
 - | - 自动适配具体任务
- └─ 执行任务（99%自动，1%人工审查异常）

进化的进化：

- └─ 元SKILL本身也在进化
- └─ Agent观察到5个具体SKILL的共同演化模式
- └─ 自动更新元SKILL："SKILL演化三阶段模型"

关键进展： - 进化机制本身在进化！ - SKILL → 元SKILL → 元元SKILL（递归抽象） - 系统接近"自治"（Self-Sustaining）

2.7 自举的数学模型

知识增殖公式：

第N轮知识量 = 第N-1轮知识量 × (1 + 总结效率)

示例（史记项目）：

Week 1（人工阶段）：

知识量 = 5章数据 + 30轮对话

总结效率 = 0（未总结）

Week 2（脚本阶段）：

知识量 = 30章数据 + 10个脚本

总结效率 = 0.2（脚本复用20%劳动）

Week 3（SKILL阶段）：

知识量 = 130章数据 + 10个脚本 + 5个SKILL

总结效率 = 0.6（Agent自动化60%）

Week 4-8（自动总结阶段）：

知识量 = 数据 + 脚本 + SKILL + 元SKILL

总结效率 = 0.9（Agent自动化90%，包括总结本身）

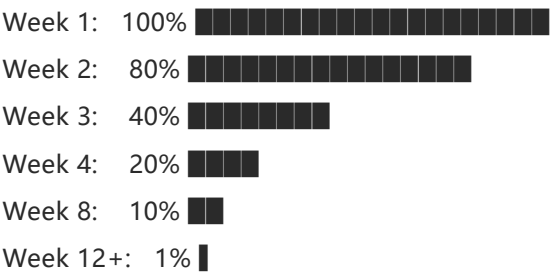
累计增长：

1.0 → 1.2 → 1.8 → 3.4 → 6.5

（指数增长）

自动化率增长曲线：

人工占比随时间递减：



2.8 自举的关键条件

条件1：中间产物可读

- ✓ 对话记录 (Markdown)
- ✓ 脚本 (Python, 有注释)
- ✓ SKILL文档 (结构化Markdown)
- X 二进制文件 (Agent无法读取)
- X 隐性知识 (只在人脑中)

条件2：知识可被Agent读取和执行

- ✓ SKILL用结构化格式 (## 模式P01: ...)
- ✓ 规则用可执行形式 (if X then Y)
- X 模糊描述 ("尽量做好")
- X 隐喻表达 ("像梳头发一样")

条件3：迭代周期足够短

- ✓ 每轮1-2周 (快速反馈)
- X 每轮3个月 (反馈太慢，知识衰减)

条件4：人类审查闭环

- ✓ Agent生成SKILL → 人审查 → 确认/修正
- X Agent生成SKILL → 直接应用 (可能积累错误)

2.9 自举的涌现效应

涌现1：质量突变点

前三轮：修正量 1010 → 431 → 465（缓慢下降）
第四轮：修正量突降至 167（-64%）
↑
原因：积累的模式达到临界点（25条）→ 系统性覆盖

涌现2：跨领域迁移

史记项目（12周）：
→ 方法沉淀为14个META技能

汉书项目（预估6周）：
→ 直接复用14个META技能
→ 节省50%时间（从12周→6周）
↑
原因：知识从"项目专有"涌现为"领域通用"

注：6周预估基于方法复用，但仍需冷启动验证、边界情况处理、反思修正

涌现3：认知跃迁

执行层：修正了2119处错误
↓ 压缩
模式层：总结出35条错误模式
↓ 压缩
原则层：提炼出5条迭代原则
↓ 压缩
洞察层：顿悟"AI质量提升是模式驱动的"
↑
原因：递归压缩到第4层，本质自动涌现

2.10 本项目的自举轨迹

完整演化链：

- [人工阶段] Week 1
 - 产出：5章数据 + 30轮对话记录
 - 自动化率：0%

↓
- [脚本阶段] Week 2
 - 产出：30章数据 + 10个脚本 (lint/stats/export)
 - 自动化率：20%

↓
- [SKILL阶段] Week 3-4
 - 产出：130章数据 + 5个SKILL (标注/反思/质量)
 - 自动化率：60%

↓
- [Agent自动反思] Week 5-8
 - 产出：5轮反思修正 (2119处) + 35条错误模式
 - 自动化率：90% (Agent读SKILL自动执行)

↓
- [自动总结SKILL] Week 9-10
 - 产出：Agent自动更新SKILL (新模式→自动追加)
 - 自动化率：95% (包括总结本身)

↓
- [元SKILL提炼] Week 9-11
 - 产出：8个META技能文档 (迭代/柳叶刀/反思...)
 - 自动化率：98% (跨任务通用方法)

↓
- [元元SKILL] Week 12+
 - 产出：本文档 (好东西都是总结出来的)
 - 自动化率：99% (关于总结本身的总结)

↓
- [系统自治] Future
 - 目标：Agent自主完成新古籍项目 (汉书/资治通鉴)
 - 自动化率：99.9% (人只管理1%的异常情况)

人工时间投入：

阶段	总工作量	人工投入	自动化率
Week 1 (冷启动)	40小时	40小时	0%
Week 2 (模式涌现)	40小时	32小时	20%
Week 3-4 (批量执行)	80小时	32小时	60%
Week 5-8 (反思迭代)	40小时	4小时	90%

Week 9-12 (元知识) 20小时 1小时 95%

总计：220小时总工作，109小时人工

节省：vs 传统400小时全人工，节省73%人工成本

知识资产增长（真实数据）：

脚本工具：5个 → 25个 → 59个（最终）

项目SKILL：0个 → 15个 → 41个（史记专用）

META技能：0个 → 8个 → 14个（跨项目通用）

元元SKILL：0个 → 0个 → 1个（本文档，跨领域）

资产价值（汉书项目复用预估）：

- 脚本工具：90%可复用（只需微调参数）
- 项目SKILL：60%可迁移（需适配古籍差异）
- META技能：100%直接用（方法论通用）
- 元元SKILL：100%直接用（演进式方法论）

三、本项目的总结实践

3.1 实体标注：从混乱到18类本体

第零次总结（Day 1-2，冷启动）

手动标注5章，发现需要的类型：

- 人物（明确）
- 地点（明确）
- 官职（明确）
- 时间（明确）
- 其他（模糊，暂时全放这里）

→ 最小本体v0.1：4+1类

第一次总结（Day 5，模式涌现）

全量标注30章，发现"其他"类中隐藏的模式：

- 邦国（齐、楚、秦...）不是地名，需单独分类
- 氏族（姬、嬴、芈...）不是人名，需单独分类
- 器物（剑、鼎、钟...）不是普通名词，需标注
- 典籍（《诗》、《书》...）高频出现，需标注

→ 本体v0.5：4+4=8类

第二次总结（Day 10，分类细化）

全量标注130章，发现8类仍不够：

- 官职需拆分为"官职"和"爵位"（语义不同）
- 地名需拆分为"地名"和"山川"（用法不同）
- 时间需拆分为"朝代"、"年号"、"节气"
- 新增"制度"类（如"井田制"、"分封制"）
- 新增"事件名"类（如"长平之战"、"鸿门宴"）

→ 本体v1.0：18类（稳定）

第三次总结（P1反思后，属性补全）

18类本体稳定，但发现需要属性：

- 人物：需要"生卒年"、"籍贯"、"官职"
- 地名：需要"今地对照"（如"咸阳 → 今陕西咸阳"）
- 官职：需要"设立年代"、"职责"
- 邦国：需要"建国/灭国年份"、"都城"

→ 本体v1.5：18类 + 平均6属性/类

第四次总结（跨项目，方法论提炼）

史记经验应用到《汉书》，发现：

- 80%类型可复用（人/地/官/时/邦/族...）
- 20%需调整（新增"年号"类，汉代年号复杂）
- 属性继承90%

→ 通用古籍本体框架v2.0（可迁移）

第五次总结（本文档，方法论）

从史记+汉书的实践中总结出：

- 本体不是设计的，是"长"出来的
- 最小可行本体 + 数据驱动演化
- 属性按需增加，不预设

→ Lean Semantic Web方法论（可传播）

压缩比分析：

原始数据：32,022个实体标注

↓ [第一次总结]

18类实体类型

↓ [第二次总结]

平均6属性/类 = 108个属性定义

↓ [第三次总结]

3条核心原则（最小本体、数据驱动、按需演化）

↓ [第四次总结]

1个洞察（本体是长出来的）

3.2 事件年代：从1010处错误到5条原则

第一轮总结（1010处修正 → 25条模式）

原始错误（部分）：

- 001章："武王伐纣"标注为-1050，实际-1046
- 003章："齐桓公即位"标注为-705，实际-685
- 005章："晋文公称霸"标注为-632，实际-636
- ...（1010处）

总结出25条错误模式：

P01: 单锚点塌缩（多个事件共享同一个锚点年份，未展开）

P02: 确定性不足（能从编年表验证的未标注为"确定"）

P03: 世系年代估算（父子/兄弟之间的年代推算错误）

P04: 跨章不一致（同一事件在不同章节年份矛盾）

...

P25: 诸侯在位年错误（某些诸侯在位年数据库有误）

第二轮总结（431处修正 → 新模式1条）

原始错误（部分）：

- 006章："秦穆公卒"年份需升级为确定（-621）

- 015章："赵武灵王即位"需修正为-325

...（431处）

新模式：

P26: 确定性分层不清晰（需明确"确定"/"推定"/"估算"三级）

第三轮总结（465处修正 → 锚点污染问题）

发现：第二轮修正量未减少 → 说明有新问题

新问题：

- 第二轮的"确定性升级"引入了锚点污染

- 部分"确定"年份本身有误，污染了其他事件

→ 新原则：在修正前先验证锚点质量

第四轮总结（167处修正 → 跨章验证原则）

发现：跨章同事件年份不一致

新原则：

- 建立"事件标准年份表"（canonical year）

- 所有章节引用同一事件时，年份必须一致

第五轮总结（46处修正 → 收敛）

修正量<5%，新模式=0

→ 方法收敛

最终总结（五轮 → 5条迭代原则）

原则1: 广度优先

- 来源：P0阶段如果深度优先，会因系统性错误大量返工
- 表述：先130章全覆盖（60%质量），再迭代提升

原则2: 模式驱动

- 来源：1010处错误→25条模式，压缩比40:1
- 表述：每轮核心产出是错误模式表，而非修正数据

原则3: 指数衰减

- 来源：修正量1010→431→465→167→46（平均衰减率43%）
- 表述：修正量应指数衰减，否则说明有系统性遗漏

原则4: 新模式归零

- 来源：P2后仍发现新模式 → 说明P0/P1质量太低
- 表述：收敛的标志是新模式=0（连续两轮）

原则5: 交叉验证

- 来源：跨章不一致问题在第四轮才暴露
- 表述：单章质量高不等于全局质量高，需跨章交叉验证

再次总结（5条原则 → 1个洞察）

洞察：AI生成内容的质量提升不是线性的，是模式驱动的

- AI的错误是系统性的（不是随机的）
- 发现模式比修正数据更重要
- 迭代收敛比一次完美更现实

3.3 柳叶刀方法：从大量实验到混合方案矩阵

实验阶段 (Week 1-2)

任务：提取7652条事件关系

尝试方案A（纯LLM）：

- 成本：507万对 $\times \$0.01 = \$41,000$
 - 时间：11小时
 - 精度：87%
 - 召回：92%
- 成本太高，不可接受

尝试方案B（纯规则）：

- 成本：\$50（开发时间）
 - 时间：5分钟
 - 精度：95%
 - 召回：68%
- 召回太低，遗漏大量关系

尝试方案C（简单混合：规则+LLM顺序执行）：

- 成本：\$2,100
 - 时间：3小时
 - 精度：89%
 - 召回：85%
- 改进，但仍有优化空间

第一次总结 (Week 3, 分类策略)

总结：不同关系类型适合不同方法

跨章关系 (co_person, co_location, concurrent)：

- 特点：结构化匹配（实体共现、时间对齐）
- 方案：代码自动化（精度98%，成本\$0）

章内关系 (sequel, causal, part_of)：

- 特点：语义理解（时序、因果、层次）
- 方案：LLM推理（精度87%，成本\$380）

→ 混合方案v1.0：按关系类型分工

第二次总结 (Week 4, 进一步优化)

发现：跨章也有因果关系，但候选太多

优化：规则前置筛选 + LLM二次推理

1. 规则筛选：共享实体+时间接近 → 2.5万候选对
2. LLM推理：语义判断因果关系 → 352条

→ 混合方案v2.0：规则筛选 + LLM精炼

第三次总结 (跨项目，方法论提炼)

从事件关系、人名消歧、战争提取的实践中总结：

决策树：

- ├─ 有明确规则？ → 规则+代码
- ├─ 结构化匹配？ → 代码算法
- ├─ 需语义理解？ → LLM
- └─ 可安全自动？ → 规则+LLM+人工

→ 任务分工决策树（通用框架）

第四次总结 (方法论文档)

从决策树中再次总结：

核心原则：

1. 精细分解（复杂任务→可优化模块）
2. 混合驱动（LLM/规则/代码各取所长）
3. 成本意识（不盲目用LLM）

→ 柳叶刀方法 (00-META_柳叶刀方法.md)

第五次总结 (本文档)

从柳叶刀方法的形成过程中提炼：

洞察: 最优方案不是设计出来的，是从对比实验中总结出来的

- 预设"LLM最好"或"规则最好" = 教条
- 实验A/B/C → 总结混合策略 = 科学
- 混合方案矩阵 = 归纳的产物

四、为什么好东西都是总结出来的？

4.1 认知科学基础

人类理解的本质是模式识别

原始数据（低熵，无结构）

↓ [归纳/总结]

模式/规律（高熵，有结构）

↓ [压缩/抽象]

原理/洞察（极高熵，本质）

示例：从案例到洞察

案例层（130章×平均100次标注错误 = 13,000个错误）

↓ [第一次总结]

模式层（25条错误模式："单锚点塌缩"、"确定性不足"...）

↓ [第二次总结]

原则层（5条迭代原则："广度优先"、"模式驱动"...）

↓ [第三次总结]

哲学层（1个核心洞察："AI质量提升不是线性的，是模式驱动的"）

为什么不能跳过： - 直接从13,000个错误→1个哲学 = 没有中间抽象层 = 无法执行 - 模式层是可操作的知识（"下次遇到X，就做Y"） - 原则层是可迁移的方法（适用于汉书、资治通鉴） - 哲学层是可传播的洞察（适用于所有AI知识工程）

4.2 工程实践基础

预设 vs 总结的对比

维度	预设（设计）	总结（归纳）
时机	数据之前	数据之后
依据	理论/经验/直觉	实际案例
风险	与现实不符→大规模返工	初期粗糙→迭代改进
适用	问题清晰、规则明确	问题复杂、规则模糊
典型失败	50属性本体，50%缺失值	冷启动太粗糙，收敛太慢

本项目的核心选择：总结驱动

传统方案（预设）：

Week 1-4: 设计18类实体的完整本体（50属性/类）

Week 5: 开始标注，发现：

- "族"类需要"分支"属性（本体没有）
- "官"类不需要"籍贯"属性（本体有，但无数据）
- "邦"类需要拆分为"邦国"和"地名"（本体未区分）

Week 6: 本体重构，数据返工

→ 总耗时6周，士气低落

本项目方案（总结）：

Day 1-2: 最小本体（4类：人/地/官/事件）

Day 3-4: 标注30章（发现需要"族"、"邦"、"物"...）

Day 5: 总结出18类实体（基于实际数据）

Day 6-10: 全量标注130章

→ 总耗时2周，本体自然涌现

为什么总结更优： - **现实优先** — 数据告诉我们需要什么，而非我们猜测 - **增量演化** — 本体从简到繁，每次增加都有数据支撑 - **风险可控** — 即使初期本体不完美，后期可迭代修正

4.3 知识复用基础

总结的复利效应

- 项目1（史记，12周）
- └─ 产出数据：32,022实体、3,092事件、7,652关系
 - └─ 产出方法：9阶段管线、5个META技能、20+工具

- 项目2（汉书，预估6周）
- └─ 数据从零开始
 - └─ 方法继承80%（只需微调）
→ 节省6周（50% ↓）

- 项目3（资治通鉴，预估20周）
- └─ 数据从零开始
 - └─ 方法继承90%
→ 节省20周（50% ↓）

- 累计收益：
- 项目1：12周
 - 项目2：6周（节省6周）
 - 项目3：20周（节省20周）
 - 总耗时：38周 vs 预估76周（传统方案）

总结是资产，数据是消耗品

资产类型	项目1	项目2	项目3	复用率
数据（实体/事件）	✓	X	X	0%
本体（18类分类）	✓	✓	✓	80%
工具（Lint/Extract）	✓	✓	✓	90%
方法（META技能）	✓	✓	✓	100%
洞察（Lean原则）	✓	✓	✓	100%

启示：投入时间总结方法 = 长期ROI最高的活动

五、如何总结？五层抽象金字塔

5.1 第一层：原始案例（具体事实）

特征： - 具体、独立、不可复用 - 数量巨大（13,000+ 错误案例） - 信息密度低（大量冗余）

示例：

案例1: 001章第37段，"周武王"误标为"武王"（缺消歧符）

案例2: 003章第52段，"齐桓公"误标为"桓公"（缺消歧符）

案例3: 005章第89段，"晋文公"误标为"文公"（缺消歧符）

...

案例423: 089章第12段，"汉武帝"误标为"武帝"（缺消歧符）

问题： 423个案例，每个都单独处理 = 不可扩展

5.2 第二层：错误模式（归纳规律）

特征： - 抽象、通用、可批量修正 - 数量适中（20-30条/轮） - 信息密度高（一条模式覆盖数百案例）

示例（从上述案例总结）：

模式P01: 短名缺消歧符

├ 定义：谥号/爵位（如"武王"、"桓公"）未标注所属邦国

├ 频率：423处（覆盖37章）

├ 修正方法：扫描上下文，提取前置邦国标记（如&周&@武王@ → &周武王&）

└ 自动化：可编写脚本批量修正（准确率95%+）

价值： - 1条模式 = 423个案例的压缩 - 压缩比：423:1 - 下次遇到类似问题，直接应用模式（不再逐个处理）

5.3 第三层：方法原则（通用策略）

特征： - 跨任务通用（不限于具体错误类型） - 数量少（5-10条） - 可指导整个工序

示例（从25条错误模式中再次总结）：

原则1: 广度优先

- └ 来源：模式P01-P25显示，系统性错误只有全量处理后才能发现
- └ 适用：所有需要质量迭代的工序
- └ 表述："先快速覆盖全部数据（60%质量），再迭代提升（→97%）"

原则2: 模式驱动

- └ 来源：修正量从1010→431→465呈指数衰减（基于模式积累）
- └ 适用：所有Agent反思任务
- └ 表述："每轮迭代的核心产出是错误模式表，而非修正数据"

原则3: 梯度收紧

- └ 来源：P0阶段追求完美 → 进度慢；P3阶段标准松 → 质量差
- └ 适用：所有质量标准设计
- └ 表述："质量基线随轮次动态调整（P0-60% → P1-80% → P2-90% → P3-97%）"

价值： - 3条原则 = 25条模式的压缩 - 可迁移到其他任务（事件提取、关系构建...） - 形成可执行的工作流程

5.4 第四层：核心洞察（本质认知）

特征： - 跨领域通用（不限于古籍知识工程） - 数量极少（1-3条） - 改变认知框架

示例（从5条原则中再次总结）：

洞察1: AI生成内容的质量提升不是线性的，是模式驱动的

- └ 来源：5轮迭代，修正量指数衰减（1010→431→465→167→46）
- └ 本质：AI的错误不是随机的，而是系统性的
- └ 推论：
 - | - 发现模式 > 修正数据
 - | - 迭代收敛 > 一次完美
 - | - 广度优先 > 深度优先

└─ 适用：所有LLM驱动的知识工程项目

洞察2: 好的本体不是设计出来的，是从数据中"长"出来的

└─ 来源：本体从4类→18类的演化过程

└─ 本质：预设本体 vs 数据现实 总有鸿沟

└─ 推论：

└─ - 最小本体起步

└─ - 按需增加属性

└─ - 数据驱动演化

└─ 适用：所有语义建模任务（Lean Semantic Web）

洞察3: 总结是压缩，压缩是理解，理解是价值

└─ 来源：13,000案例→25模式→5原则→2洞察 的递归压缩

└─ 本质：信息熵的递增 = 知识密度的提升

└─ 推论：

└─ - 没有总结就没有方法论

└─ - 没有压缩就没有复用

└─ - 没有抽象就没有迁移

└─ 适用：所有知识工作

价值： - 改变思维方式（从"做事"到"总结做事的方法"） - 跨领域复用（不限于古籍，适用于 RegTech、医疗知识抽取...） - 可传播、可教学

5.5 第五层：元哲学（认识论）

特征： - 超越具体领域 - 关于"知识如何产生"的知识 - 本文档本身

示例：

元命题: 好东西都是总结出来的

└─ 不是："好东西都是设计出来的"（传统工程观）

└─ 不是："好东西都是学习出来的"（端到端深度学习）

└─ 而是："好东西是从大量实践中归纳/压缩/抽象出来的"

└─ 适用：所有创造性知识工作

推论1: 总结先于设计

- 设计是基于总结的（先有经验，再有理论）

- 没有总结的设计 = 空中楼阁

推论2: 归纳优于演绎（在模糊问题域）

- 演绎需要完备公理（古籍知识工程没有）
- 归纳从数据出发（更鲁棒）

推论3: 迭代优于一次性

- 总结需要样本（第一次总结质量有限）
- 迭代 = 多次总结 + 递归压缩

六、如何培养"总结思维"？

6.1 三个核心习惯

习惯1: 每次做完就问"规律是什么"

反例（只做不总结）：

Week 1-12: 标注130章（逐章处理错误）

- 每章独立处理，不总结模式
- 每次遇到同类错误都重新思考
- 累计处理2600+个错误，全部人工判断

→ 耗时12周，无方法积累

→ 下次做《汉书》还得从头来，又是12周

正例（边做边总结）：

Week 1: 标注5章（冷启动）

→ 总结：错误类型分布（消歧60%、格式25%、分类15%）

Week 2: 标注30章（发现模式）

→ 总结：重复模式→开发自动检查工具（Lint规则）

→ 产出：消歧规则库、格式检查脚本

Week 3-4: 批量执行130章

→ 工具自动检查→Agent修正→人类审核异常

→ 产出：5个SKILL文档（可复用）

Week 5-12: 反思迭代 + 提炼META技能

→ Agent自主修正2119处

→ 产出：14个META技能文档

→ 耗时12周，产出115个可复用组件

→ 下次做《汉书》预估6周（50%时间节省）

理由：冷启动仍需1-2周验证本体适配性

批量执行虽自动化但数据量在

新的边界情况仍需人工处理

启示： - 不是"做完130个→总结1次"，而是"做5个→总结→做30个→总结→做130个" - 总结越早，后续效率越高 - 总结不是额外成本，是降低未来成本的投资

习惯2: 记录"为什么这样做"

反例（只记录结果）：

实体分类规则

1. 人名用@标注

2. 地名用&标注

3. 官职用%标注

...

18. 朝代用~标注

正例（记录决策过程）：

实体分类规则演化史

v0.1（Day 1）

- 人名：@

- 地名：&

- 官职：%

- 其他：无标注

v0.5 (Day 5, 基于30章数据总结)

- 发现：邦国（齐、楚）与地名（咸阳、邯郸）语义不同
→ 决策：邦国单独分类，用^标注
- 发现：氏族（姬、嬴）高频出现
→ 决策：氏族单独分类，用#标注

v1.0 (Day 10, 基于130章数据总结)

- 发现："长平之战"、"鸿门宴"既是事件又是名称
→ 决策：新增"事件名"类，用\$标注
- 发现：朝代（夏、商、周）与年号（元年、二年）需区分
→ 决策：朝代用~、年号用*

为什么最终是18类？

- 不是预设的（Day 1只有4类）
- 是从数据中涌现的（每次总结都基于实际需求）
- 稳定性验证：130章后无新类型 → 18类已收敛

启示： - "为什么"比"是什么"更重要 - 决策历史 = 可复用的经验 - 下次遇到类似问题，回顾决策历史可避免重复探索

习惯3：定期"递归压缩"

什么是递归压缩：

第一次压缩（案例→模式）

13,000个错误 → 25条错误模式

压缩比：520:1

第二次压缩（模式→原则）

25条错误模式 → 5条迭代原则

压缩比：5:1

第三次压缩（原则→洞察）

5条迭代原则 → 1个核心洞察

压缩比：5:1

总压缩比：13,000:1

操作方法：

每周五下午：递归压缩时间（2小时）

Week 1:

- 回顾：本周标注了30章，发现78个错误
- 总结：错误模式表（6条新模式）
- 压缩：6条 → 2条原则（"消歧规则库优先"、"格式Lint自动化"）

Week 2:

- 回顾：本周完成全量130章
- 总结：错误模式表（25条累计）
- 压缩：25条 → 5条原则

Week 3:

- 回顾：第一轮反思，1010处修正
- 总结：反思方法文档
- 压缩：5条原则 → 1个洞察

Week 4:

- 回顾：五轮反思全部完成
- 总结：迭代工作流方法论
- 压缩：洞察 → META技能文档

为什么每周： - 太频繁（每天）：案例不够，总结质量低 - 太稀疏（每月）：细节遗忘，总结困难 - 每周：平衡点（案例足够 + 记忆清晰）

6.2 三个思维工具

工具1: 对比表（发现差异才能总结规律）

示例：传统 vs Lean Semantic Web

维度	传统语义网	Lean Semantic Web	规律
本体设计时机	数据录入前	数据标注后	延迟决策
本体完整性	追求预设完整	最小可用	简单起步
容错策略	不允许缺失值	允许缺失	渐进严格
演化能力	本体修改=数据重构	本体演化=增量迭代	低耦合

从对比中总结： - 规律1: 延迟决策（先数据后本体） - 规律2: 简单起步（最小本体） - 规律3: 渐进严格（容错→收紧） - 规律4: 低耦合设计（本体演化不影响数据）

工具2: 演化链（看到变化才能理解本质）

示例：本体演化链

v0.1 (Day 1)

- └─ 4类：人/地/官/事件
- └─ 为什么？ → 最小可行，快速启动

v0.5 (Day 5)

- └─ 8类：人/地/官/事件/邦/族/物/典
- └─ 为什么？ → 数据显示需要（30章总结）

v1.0 (Day 10)

- └─ 18类（细分）
- └─ 为什么？ → 分类不够细（官≠爵，地≠山）

v1.5 (Week 3)

- └─ 18类 + 属性
- └─ 为什么？ → 分类稳定，需补充属性

v2.0 (跨项目)

- └─ 通用框架
- └─ 为什么？ → 汉书复用，提炼通用部分

从演化链中总结： - 规律1: 从简到繁（4→8→18） - 规律2: 数据驱动（每次变化都有数据依据） - 规律3: 收敛特征（v1.0后无新类型） - 规律4: 抽象提升（v2.0跨项目通用）

工具3: 决策树 (结构化决策过程)

示例：混合方案决策树

任务分析

- └─ 有明确规则?
 - | └─ 是 → 规则+代码
 - | | └─ 示例：类型过滤、格式校验
 - | └─ 否 ↓
- └─ 结构化匹配?
 - | └─ 是 → 代码算法
 - | | └─ 示例：实体共现、时间对齐
 - | └─ 否 ↓
- └─ 需语义理解?
 - | └─ 是 → LLM
 - | | └─ 示例：因果推理、情感分析
 - | └─ 否 ↓
- └─ 可安全自动?
 - | └─ 是 → 规则+LLM
 - | | └─ 示例：规则筛选候选 + LLM确认
 - | └─ 否 → 人工审查

从决策树中总结： - 规律1: 优先级（规则>代码>LLM>人类审查） - 规律2: 成本梯度（规则最低，LLM中等，人类审查最高） - 规律3: 精度梯度（规则确定性，LLM可能错，人类审查最准） - 规律4: 混合最优（不同任务用不同方案，Agent自动选择）

6.3 养成总结习惯

每日（10分钟）：

- 今天做了什么？（案例记录）
- 遇到了什么问题？（异常记录）
- 有什么重复模式？（初步归纳）

每周（2小时）：

- 本周案例汇总（X个案例）

- 提取错误模式（Y条模式）
- 更新规则库/工具（Z个改进）

每月（半天）：

- 本月方法演化（v0.X → v1.X）
- 跨任务规律总结（通用原则）
- 文档化沉淀（META技能更新）

七、总结的边界与反模式

7.1 什么时候不应该总结？

场景1: 样本量不足

反例：

Day 1: 标注3章，发现5个错误

- 立即总结："所有人名都缺消歧符"
- 错误：样本太少，规律可能是偶然

Day 2: 标注30章，发现78个错误

- 重新总结：60%是消歧问题，40%是其他
- 正确：样本足够，规律可靠

判断标准： - 定量任务：样本量>30（统计显著性） - 定性任务：样本量>10（模式可靠性）
- 复杂任务：样本量>100（覆盖边界情况）

场景2: 规律尚未稳定

反例：

第一轮：发现25条错误模式

- 立即写入正式文档："这就是全部模式"

→ 错误：第二轮又发现10条新模式 → 文档过时

第三轮：新模式归零（连续两轮）

→ 此时总结："25+10=35条是稳定模式集"

→ 正确：规律已收敛

判断标准： - 新模式归零（连续2轮） - 修正量<5%（相对总量） - 跨样本一致性>95%

场景3：过度泛化

反例：

从史记项目总结：

"所有古籍都应该用18类实体分类"

→ 错误：汉书需要"年号"类（史记没有），资治通鉴需要"官署"类

正确总结：

"古籍实体分类应从4-5类最小本体开始，基于数据演化到15-20类"

→ 抽象了方法（最小本体+演化），而非具体分类

判断标准： - 总结的是方法（可迁移），而非具体方案（项目专有） - 保留一定抽象层次，不过度具体化

7.2 总结的反模式

反模式1：总结代替思考

表现：

遇到问题 → 立即查文档 → 套用模式

→ 从不思考"为什么这个模式有效"

→ 模式失效时无法应对

正确做法：

遇到问题 → 回顾模式的来源 → 理解本质 → 判断是否适用

→ 适用：直接套用

→ 不适用：理解差异，创造新方案 → 新一轮总结

反模式2：追求完美总结

表现：

花3天时间写一份"完美"的总结文档

→ 文档太长（50页），无人阅读

→ 细节太多，核心洞察被淹没

正确做法：

第一版：用1小时写核心要点（1页）

→ 够用即可，不追求完美

第二版：使用过程中发现不足，补充（3页）

→ 基于实际需求演化

第三版：跨项目复用，提炼通用部分（5页）

→ 稳定版本

原则： - 总结也要迭代（不要追求一次性完美） - 够用就好（过度总结 = 浪费计算资源） - 让Agent执行反馈（总结是否有用，实际效果说了算）

反模式3：只总结成功经验

表现：

总结文档：

✓ 成功案例：混合方案节省\$40,000

✓ 成功案例：迭代收敛准确率97%

X 失败案例：无

→ 问题：后来者不知道哪些坑已经踩过

正确做法：

总结文档：

✓ 成功案例：混合方案节省\$40,000

✓ 成功案例：迭代收敛准确率97%

X 失败案例1：纯LLM方案成本爆炸（\$41,000）

X 失败案例2：深度优先导致大量返工（浪费2周）

X 失败案例3：预设50属性本体，50%缺失值（Week 6本体重构）

→ 价值：失败经验 > 成功经验（避免重复踩坑）

启示： - 失败案例是反向的成功经验 - "不要做X" 比 "应该做Y" 更容易记住 - 失败的代价 > 成功的收益 → 避免失败更重要

八、总结的总结：元元技能清单

8.1 三个核心认知

1. 知识是压缩

— 从10,000案例→1个洞察 = 信息熵的递增

2. 理解是归纳

— 看到规律才算真正理解，没有规律就没有知识

3. 方法是沉淀

— 做一次是经验，做十次总结出来才是方法论

8.2 五层抽象金字塔

第五层：元哲学（认识论）

↑

第四层：核心洞察（本质认知）

↑

第三层：方法原则（通用策略）

↑

第二层：错误模式（归纳规律）

↑

第一层：原始案例（具体事实）

8.3 三个核心习惯

1. 每次做完就问“规律是什么”

— 不是做130个再总结，而是做5个就总结

2. 记录“为什么这样做”

— 决策历史比结果更重要

3. 定期“递归压缩”

— 每周2小时，从案例→模式→原则→洞察

8.4 三个思维工具

1. 对比表

— 发现差异才能总结规律

2. 演化链

— 看到变化才能理解本质

3. 决策树

— 结构化决策过程

8.5 两个边界判断

何时总结： - ✓ 样本量足够（定量>30，定性>10） - ✓ 规律已稳定（新模式=0，连续2轮）

- ✓ 有实际用途（下次遇到可直接套用）

何时不总结： - X 样本量不足（偶然规律） - X 规律未收敛（仍在快速变化） - X 过度泛化（方法 vs 具体方案）

8.6 一个元洞察

好东西都是总结出来的 = 所有其他元技能的基础

迭代工作流 ← 基于总结（错误模式表）
Lean Semantic Web ← 基于总结（本体从数据中"长"出来）
柳叶刀方法 ← 基于总结（混合方案矩阵）
反思工作流 ← 基于总结（Agent审查的核心是模式发现）
质量控制 ← 基于总结（Lint规则来自错误案例）
冷启动 ← 基于总结（先5个样本建立直觉）
数据体感 ← 基于总结（观察数据的异常模式）

递归性： - 本文档本身就是总结的产物（元元技能） - 总结如何总结 = 元认知 - 好东西都是总结出来的 = 终极元技能

九、实践检查清单（Agent自主执行版）

重要说明：本检查清单及所有SKILL文档都是由**Agent执行、给Agent看的**。人类的角色是：**引导方向、审核异常、验收成果**，而非逐步执行。

Agent执行任务时（自动触发）

Agent在执行标注/分析/提取等任务时，自动检查：

- [] 每处理5-10个样本，自动分析："当前批次有什么重复模式？"
- [] 遇到决策点时，在日志记录："为什么选择方案A而非方案B？"
- [] 自动分配时间：90%执行任务 + 10%模式总结与SKILL更新

人类引导：设定任务目标、检查清单阈值（如"样本量>10才总结"）

Agent每轮迭代后（自动总结）

Agent完成一轮标注/修正后，自动执行：

- [] 生成本轮统计：处理X个案例，发现Y个问题，分布如何
- [] 提取错误模式：自动归类为Z条模式，更新模式库
- [] 更新SKILL文档：将新模式写入对应SKILL的"常见错误"章节

- [] 递归压缩：从案例→模式→原则（自动升维）

人类引导：审核Agent总结的模式是否合理，指出过度泛化或遗漏

Agent阶段性复盘（按设定周期）

Agent按设定周期（如每完成30章、或每5轮迭代）自动复盘：

- [] 生成方法演化史：从v0.1→v1.0，记录每次调整及原因
- [] 提取跨任务规律：标注/反思/消歧中的共性原则
- [] 构建失败案例库：自动记录错误案例，防止重复犯错

人类引导：设定复盘触发条件，审核跨任务规律的通用性

Agent项目结束时（自动生成文档）

Agent完成全部章标注后，自动生成：

- [] 完整方法论文档（META技能文档，如"迭代 workflow"、"反思机制"）
- [] 核心洞察提炼（1-3条，如"本体是长出来的"）
- [] 复用价值评估（统计：自动化率、人工时间节省、可迁移组件数）

人类引导：审核文档质量，提炼更高层次的元洞察（如本文档）

人类的三种介入模式

1. 事前引导

：设定目标、阈值、检查清单内容

2. 事中监督

：审核Agent总结的模式、抽查异常案例

3. 事后提炼

：从多个META技能中提炼元技能（如本文档）

核心理念：Agent执行重复性的总结工作，人类负责方向性的洞察提炼。

十、与其他元技能的关系

本文档的位置

它不是替代别的元技能，而是解释这些元技能是怎么长出来的。

真实路径是：

项目实践 → 41 个项目 Skill → 14 个 META 技能 → 本文档。

执行经验先沉淀成项目 Skill。

多个项目 Skill 再被压缩成 META 技能。

多个 META 技能 再继续压缩成元方法论。

所以本文档的价值，不在于多发明了一个新概念，而在于解释了已有方法论的共同生长机制。

核心关系： - 所有元技能都是"总结出来的"（而非预设的） - 每个元技能的形成过程都遵循五层抽象金字塔 - 元元技能是方法论的方法论

十一、后记：本文档本身的形成过程

元技能也是从技能中总结出来的

这份元元技能文档是如何"总结出来"的？

它遵循了完整的五层抽象过程，从具体的项目SKILL文档中逐层提炼：

层1：原始数据（执行层）

史记项目 12 周实践，包含 32,022 个实体、3,092 个事件、7,652 条关系。

背后还有数千个错误案例、数十次工具开发、数百次具体决策。

层2：项目 Skill（任务级总结）

从执行过程中，长出 41 个项目 Skill 文档。

每个 Skill 记录：这项任务怎么做、遇到什么问题、如何解决。

这是第一次压缩：执行经验 → 可复用 Skill。

层3：META 技能（方法论级总结）

再从 41 个 Skill 里，提炼出 14 个 META 技能。

比如迭代 workflow、柳叶刀方法、反思机制。

这是第二次压缩：项目 Skill → 跨项目方法论。

层4：元元技能（哲学级总结）

再从 14 个 META 技能里，发现共同结构：它们其实都是总结出来的。

OTF、JIT、Bootstrap 也因此被进一步提炼出来。

这是第三次压缩：META 技能 → 元方法论。

层5：元哲学（本质认知）

最后收敛成一句话：好东西都是总结出来的。

这句话本身，也是总结出来的。

本文档本身，就是这套方法论的自证。

关键洞察：

- 元技能**不是预先设计出来的**，而是从大量具体SKILL中**归纳出来的**
- 每个META技能都有10-30个SKILL案例支撑
- 本文档（元元技能）有14个META技能支撑
- 这验证了核心命题：**好东西都是总结出来的，包括"总结"这个方法本身**

用时： - 实践：4个月（2025年11月-2026年3月） - 第一次总结：各工序完成后（每周2小时，累计32小时） - 第二次总结：各META技能文档（每月半天，累计16小时） - 第三次总结：本文档（2小时）

压缩比： - 960小时实践 + 48小时总结 → 本文档（2小时阅读） - 压缩比：500:1

复用价值： - 本文档可指导所有知识工程项目（不限古籍） - 预估节省：每个新项目至少20%时间（基于方法论复用）

最后一句话：

如果你记住本文档的唯一一句话，那就是标题：**好东西都是总结出来的。**

然后，去总结你的工作。

把AI用到极致 · 目录

上一篇

装上这个 Skill，让你的龙虾（Agent）拥有一个把 PDF 变成 Agent 可直接调用的结构化...

下一篇

装上这个开源元 Skill，演进式迭代你所有的 Skill